

ML

Linear data

DT / XGBoost / etc

Non linear data

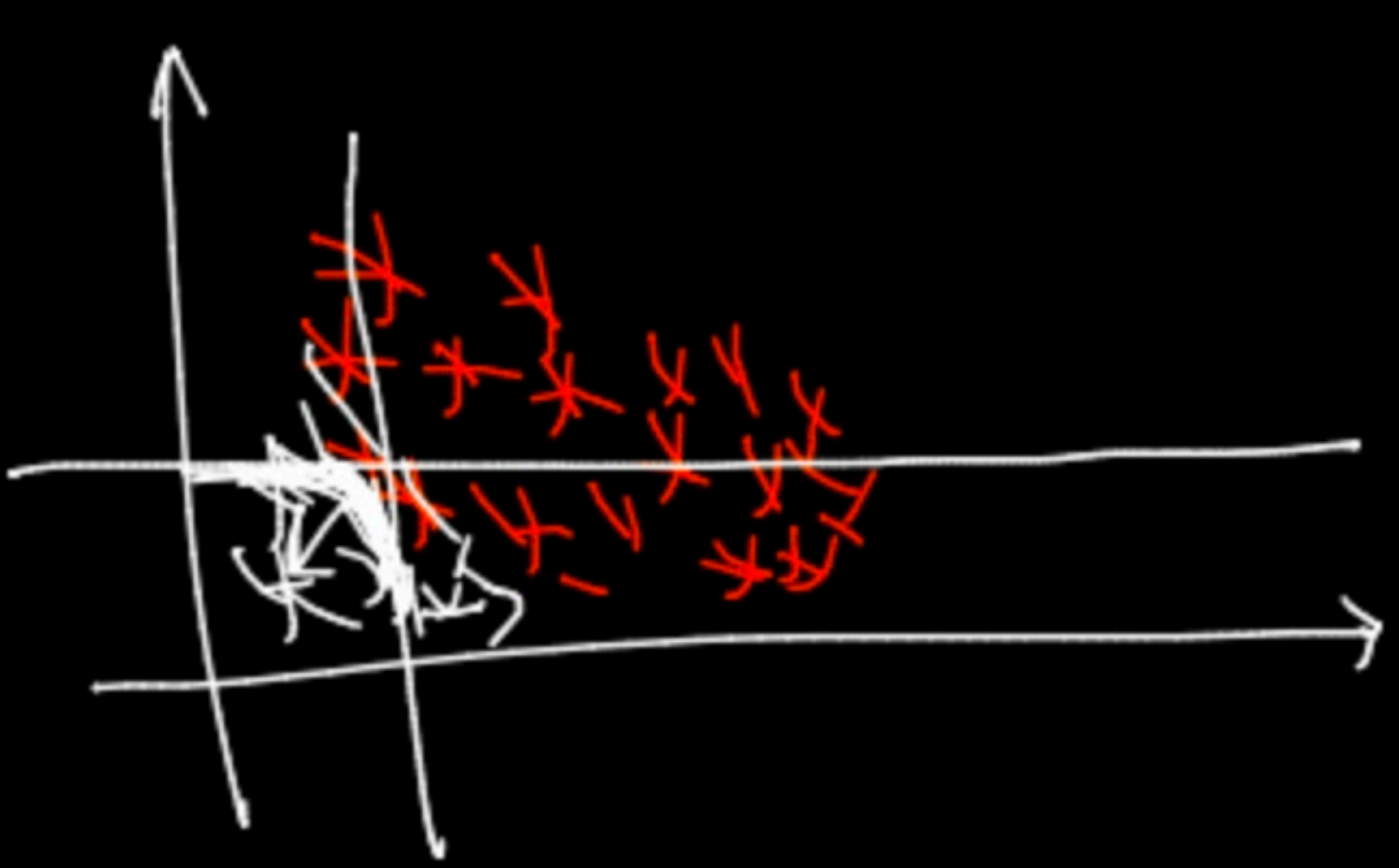
Deep learning

NN

eg.

Non linear

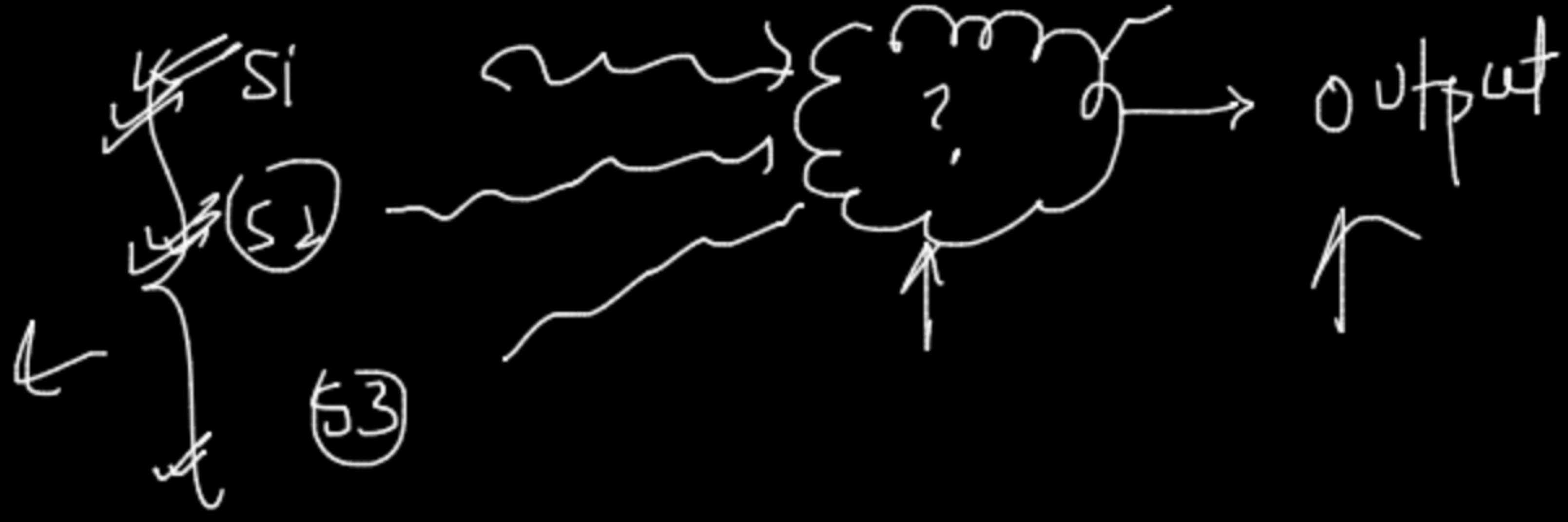
Complex

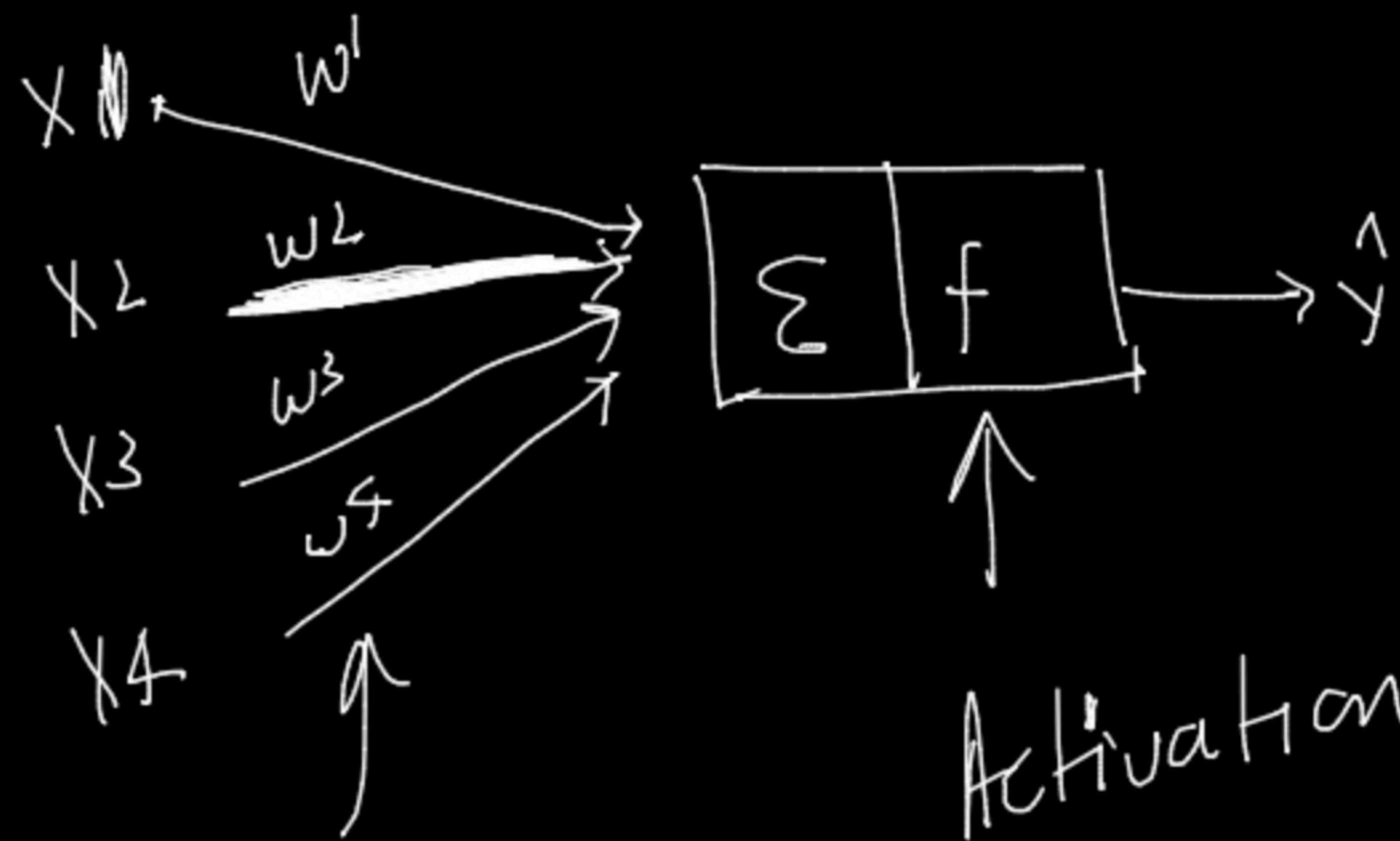


Linear



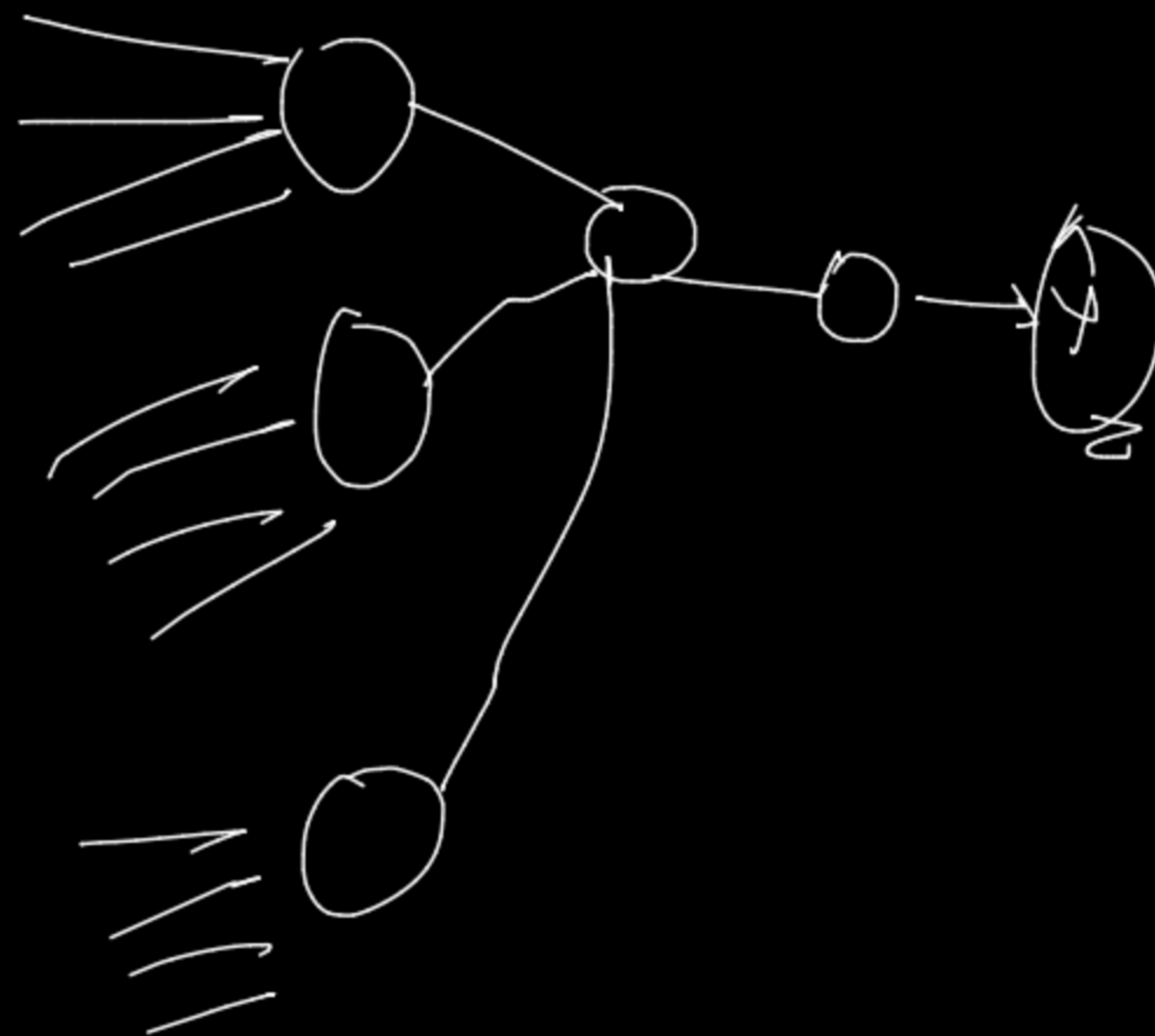
inputs





Activation function

Human brain



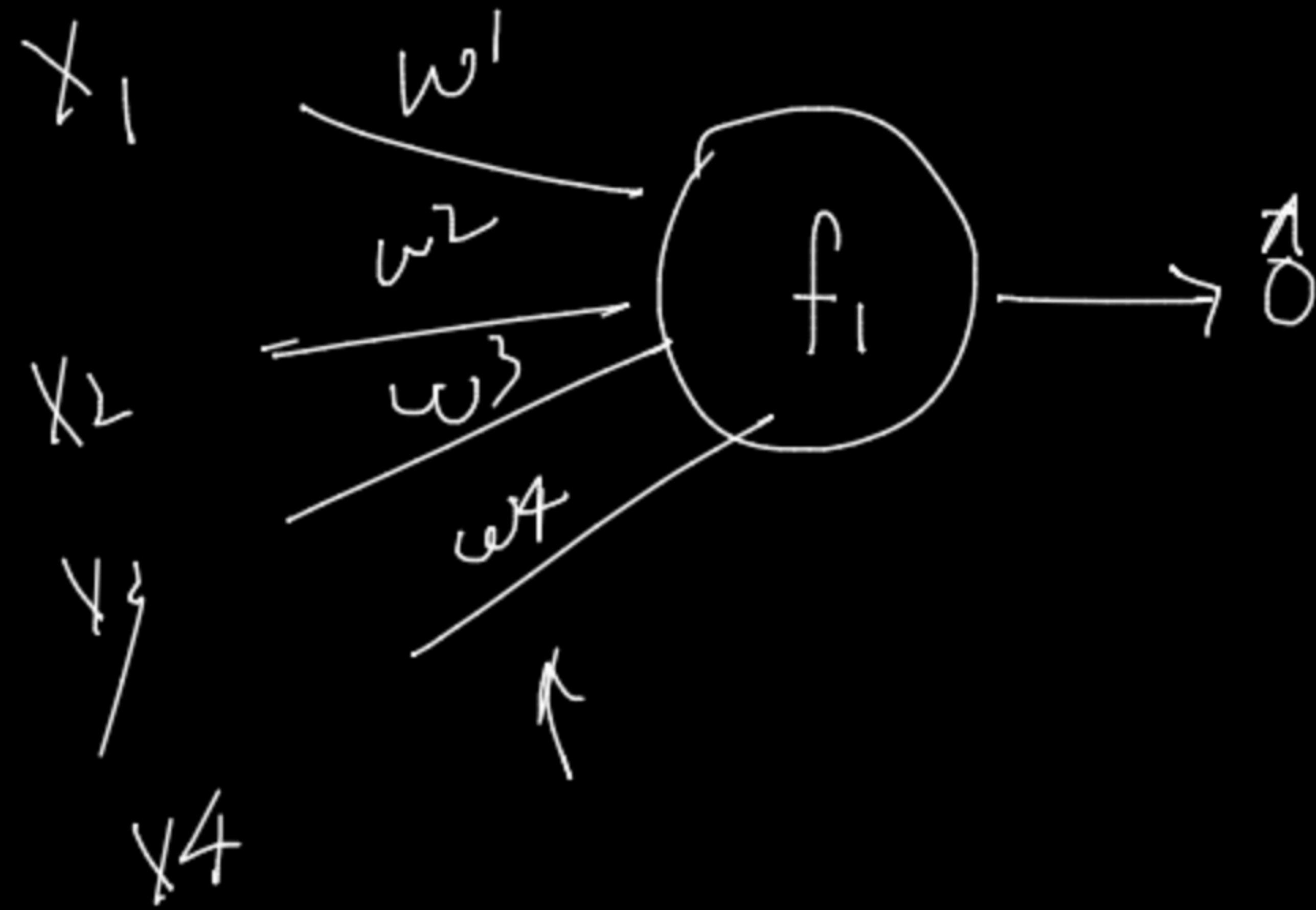
weights

Neuron

simple

1457

perception → NN
↑
logistic Regression
↑



mathematically

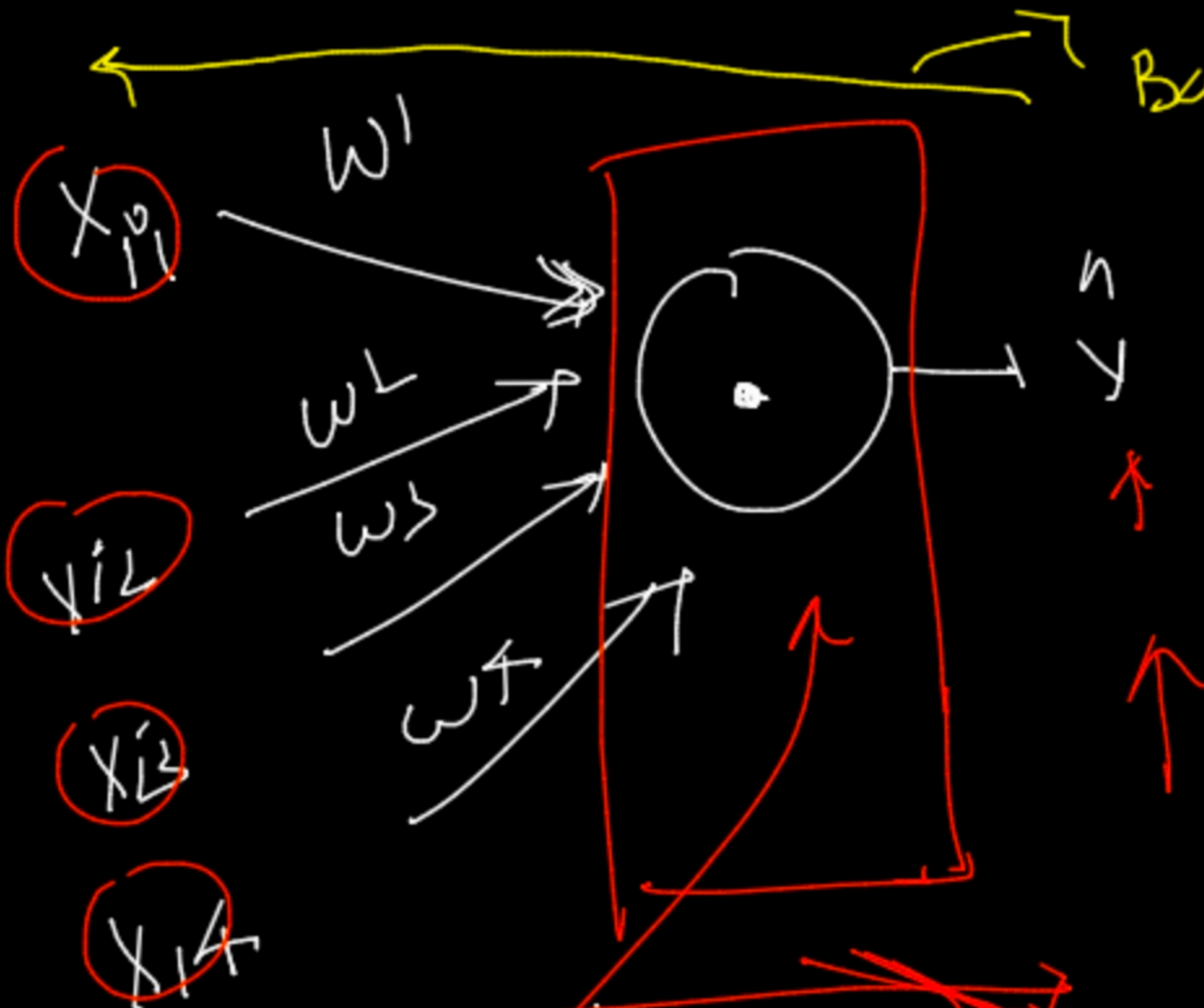
↑

$$\hat{0} = f \left(\sum_{i=1}^n x_i w_i \right)$$

input ← x_i weight ← w_i

ML

Backpropagation



y_i

f_1	f_2	f_3	f_4	x	y
-	-	-	-	0	0
-	-	-	-	1	1
-	-	-	-	0	0

1 epochs

→ feed forward

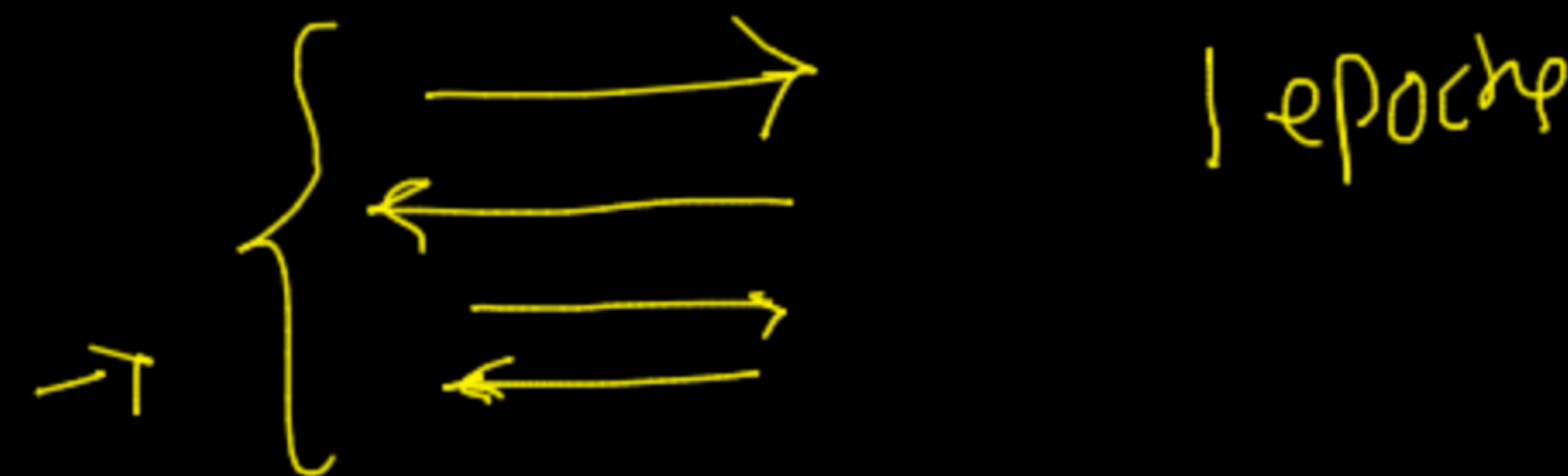
$[0, 1, 0, 1]$ → 80%

$$\hat{y} = f(x_{i1} * w_1 + x_{i2} * w_2 + x_{i3} * w_3 + x_{i4} * w_4)$$

logistic

$$\hat{y} = f\left(\sum_{i=1}^d w_i^T x_i\right) \rightarrow \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

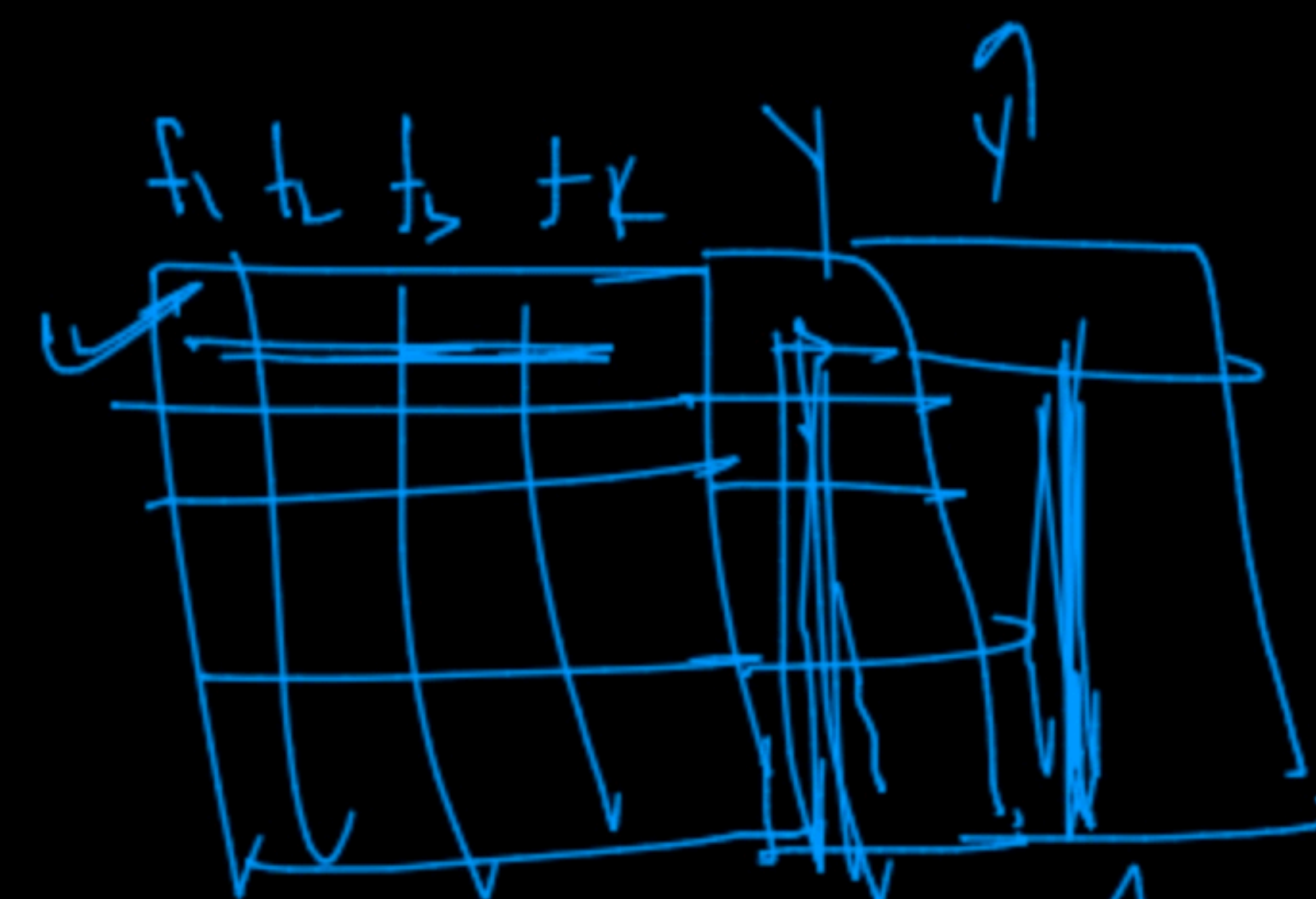
↑
sigmoid



MLP (Multi layer perceptron)

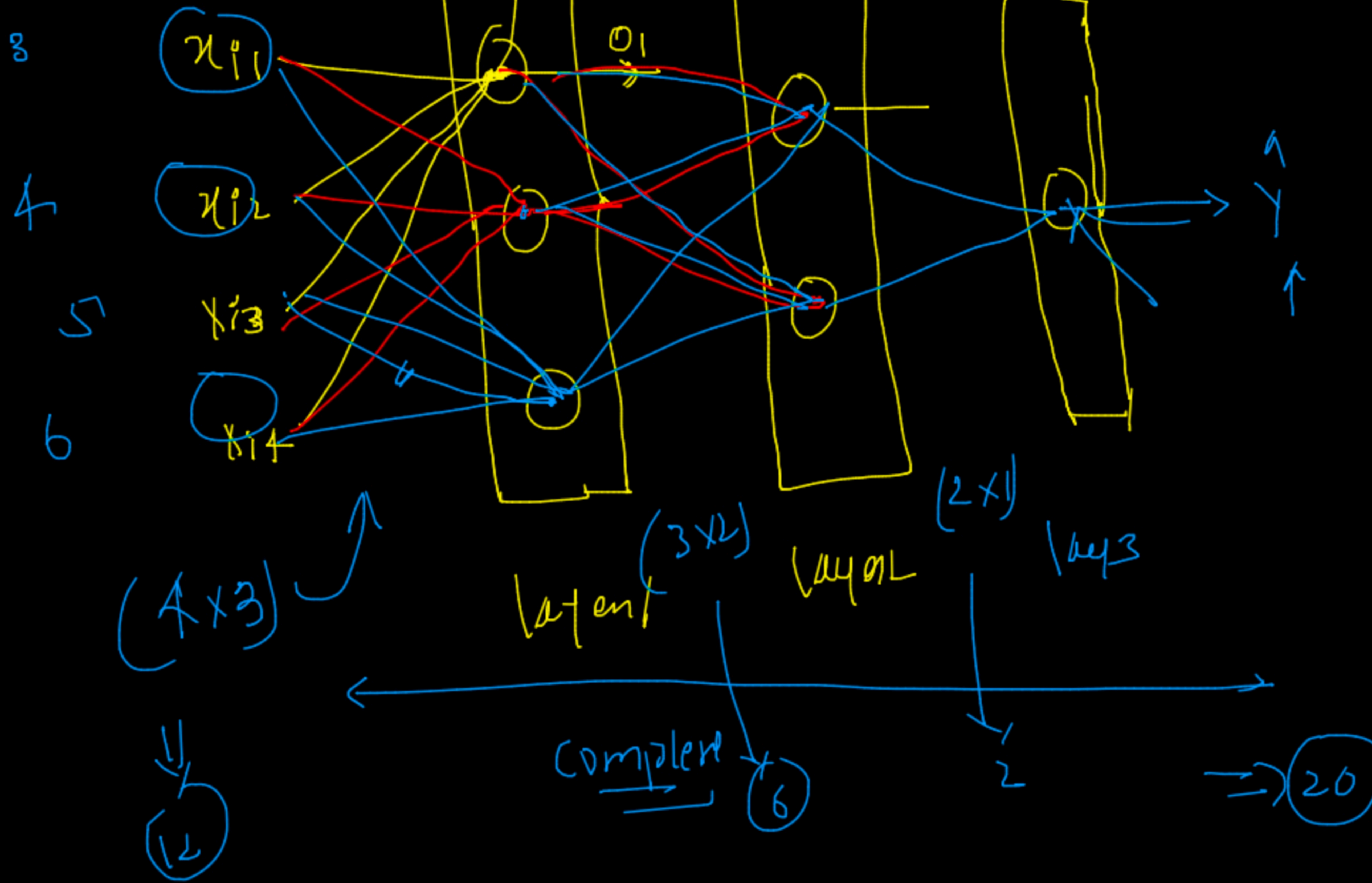
Activation

Backpropagation



Compu

error = 5.6



act (layer 1)

$$= \begin{bmatrix} w_1 & w_2 & \dots \\ \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \end{bmatrix}$$

↑

Random weights

$$\begin{bmatrix} 3, 2, 1, 4 \end{bmatrix} \begin{bmatrix} 0.4 \\ 0.1 \\ 0.2 \\ 0.1 \end{bmatrix}$$

$$\underline{3 \times 0.4 + 2 \times 0.1 + 0.2 \times 1 + 4 \times 0.1}$$

=

